

# Algoritmos de Alocação: Sequencial e Otimizado

Sistema SIVIRA - Otimização de Produção

Documentação Técnica

Dezembro 2025

## Sumário

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>Introdução</b>                                           | <b>3</b> |
| 1.1      | Objetivos do Sistema . . . . .                              | 3        |
| <b>2</b> | <b>Algoritmo Sequencial</b>                                 | <b>3</b> |
| 2.1      | Descrição . . . . .                                         | 3        |
| 2.2      | Arquitetura do Algoritmo Sequencial . . . . .               | 4        |
| 2.3      | Pseudocódigo . . . . .                                      | 4        |
| 2.4      | Características . . . . .                                   | 4        |
| 2.5      | Componentes do Algoritmo Sequencial . . . . .               | 5        |
| 2.6      | Vantagens e Desvantagens . . . . .                          | 5        |
| <b>3</b> | <b>Otimizador PL</b>                                        | <b>5</b> |
| 3.1      | Arquitetura do Otimizador PL . . . . .                      | 5        |
| 3.2      | Componentes do Otimizador PL . . . . .                      | 6        |
| 3.3      | Conceito de $\tau_{max}$ (tempo_maximo_de_espera) . . . . . | 6        |
| <b>4</b> | <b>Formulação Matemática</b>                                | <b>6</b> |
| 4.1      | Variáveis de Decisão . . . . .                              | 6        |
| 4.2      | Função Objetivo . . . . .                                   | 7        |
| 4.3      | Restrições . . . . .                                        | 7        |
| 4.3.1    | Restrição 1: Unicidade por Pedido . . . . .                 | 7        |
| 4.3.2    | Restrição 2: Permutação Única (Ordenação) . . . . .         | 7        |
| 4.3.3    | Restrição 3: Tempo Máximo de Espera . . . . .               | 7        |
| 4.3.4    | Restrição 4: Conflitos Temporais (Equipamentos) . . . . .   | 8        |
| 4.3.5    | Restrição 5: Deadline Rígido . . . . .                      | 8        |
| <b>5</b> | <b>Backward Scheduling</b>                                  | <b>8</b> |
| 5.1      | Modo Determinístico ( $\tau_{max} = 0$ ) . . . . .          | 8        |
| 5.2      | Modo Flexível ( $\tau_{max} > 0$ ) . . . . .                | 8        |
| 5.3      | Representação Visual . . . . .                              | 8        |
| 5.4      | Estratégia de Retry (Modo Flexível) . . . . .               | 9        |
| <b>6</b> | <b>Complexidade Computacional</b>                           | <b>9</b> |

|           |                                              |           |
|-----------|----------------------------------------------|-----------|
| <b>7</b>  | <b>Comparação: Sequencial vs Otimizado</b>   | <b>9</b>  |
| <b>8</b>  | <b>Implementação</b>                         | <b>9</b>  |
| 8.1       | Exemplo de Uso - Sequencial . . . . .        | 9         |
| 8.2       | Exemplo de Uso - Otimizador PL . . . . .     | 10        |
| 8.3       | Parâmetros de Configuração . . . . .         | 10        |
| <b>9</b>  | <b>Quando Usar Cada Algoritmo</b>            | <b>11</b> |
| 9.1       | Use o Algoritmo Sequencial quando: . . . . . | 11        |
| 9.2       | Use o Otimizador PL quando: . . . . .        | 11        |
| <b>10</b> | <b>Conclusão</b>                             | <b>11</b> |

# 1 Introdução

Este documento descreve os dois algoritmos de alocação de pedidos disponíveis no sistema SIVIRA:

1. **Algoritmo Sequencial:** Execução com heurística de priorização
2. **Otimizador PL:** Otimização unificada via MILP que suporta ambos os modos:
  - Modo DETERMINÍSTICO ( $\tau_{max} = 0$ ): Atividades contíguas
  - Modo FLEXÍVEL ( $\tau_{max} > 0$ ): Janelas temporais flexíveis

O Otimizador PL utiliza o solver **OR-Tools CP-SAT** do Google, que oferece alta performance para problemas de satisfação de restrições e otimização combinatória.

## 1.1 Objetivos do Sistema

- Maximizar o número de pedidos atendidos dentro dos prazos (deadlines)
- Respeitar restrições de equipamentos (capacidade finita)
- Garantir sequenciamento correto de atividades (gaps temporais)
- Otimizar a ordem de execução considerando urgência dos pedidos

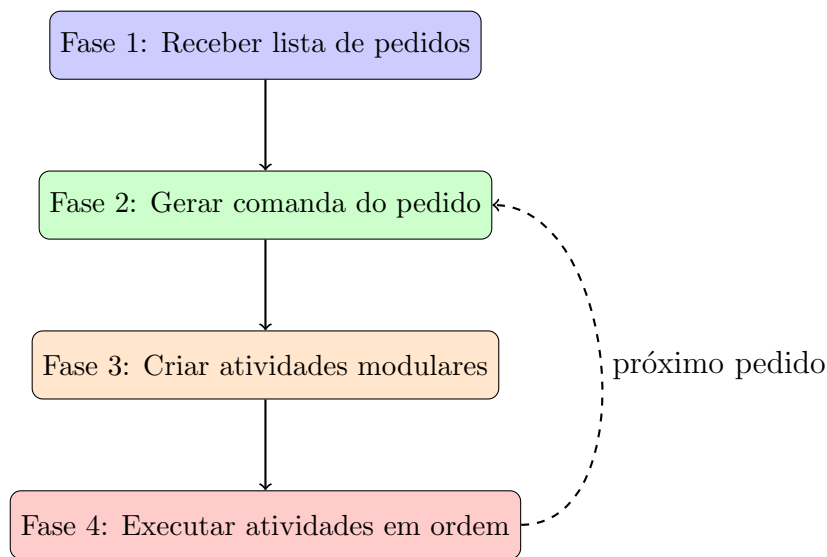
# 2 Algoritmo Sequencial

O Algoritmo Sequencial é a abordagem mais simples para execução de pedidos, utilizando uma **heurística de priorização** baseada em critérios de restrições.

## 2.1 Descrição

O método sequencial ordena os pedidos por uma heurística de priorização antes da execução, considerando: (1) duração total estimada (maiores primeiro), (2) quantidade (maiores primeiro), (3) tempo de antecipação (menores primeiro) e (4) ID do pedido como desempate. É uma abordagem **gulosa (greedy)** que processa cada pedido individualmente, sem otimização global de recursos.

## 2.2 Arquitetura do Algoritmo Sequencial



## 2.3 Pseudocódigo

---

**Algorithm 1** Execução Sequencial de Pedidos

---

```
1:  $pedidos\_executados \leftarrow 0$ 
2:  $pedidos\_com\_erro \leftarrow 0$ 
3: for cada  $pedido$  em  $lista\_pedidos$  do
4:   Gerar comanda com reserva de ingredientes
5:   Criar atividades modulares necessárias
6:   Executar atividades em ordem cronológica
7:   if execução bem-sucedida then
8:      $pedidos\_executados \leftarrow pedidos\_executados + 1$ 
9:     Salvar log de sucesso
10:  else
11:     $pedidos\_com\_erro \leftarrow pedidos\_com\_erro + 1$ 
12:    Salvar log de erro
13:    Continuar para próximo pedido
14:  end if
15: end for
16: return estatísticas de execução
```

---

## 2.4 Características

- **Ordem de execução:** Fixa, baseada na ordem de chegada (1, 2, 3...)
- **Conflitos de equipamentos:** Não são detectados previamente
- **Gaps temporais:** Não são verificados antecipadamente
- **Função objetivo:** Nenhuma (execução simples)
- **Método de solução:** Greedy (guloso)

- **Complexidade:**  $O(n)$  onde  $n$  é o número de pedidos

## 2.5 Componentes do Algoritmo Sequencial

| Módulo                               | Função                              |
|--------------------------------------|-------------------------------------|
| <code>executor_pedidos.py</code>     | Orquestrador da execução sequencial |
| <code>gestor_comandas.py</code>      | Geração de comandas e reservas      |
| <code>atividades_modulares.py</code> | Criação de atividades por pedido    |
| <code>gestor Equipamentos.py</code>  | Alocação de equipamentos            |

Tabela 1: Módulos do sistema Sequencial

## 2.6 Vantagens e Desvantagens

### Vantagens:

- Implementação simples e direta
- Baixo custo computacional
- Fácil de depurar e manter
- Comportamento previsível

### Desvantagens:

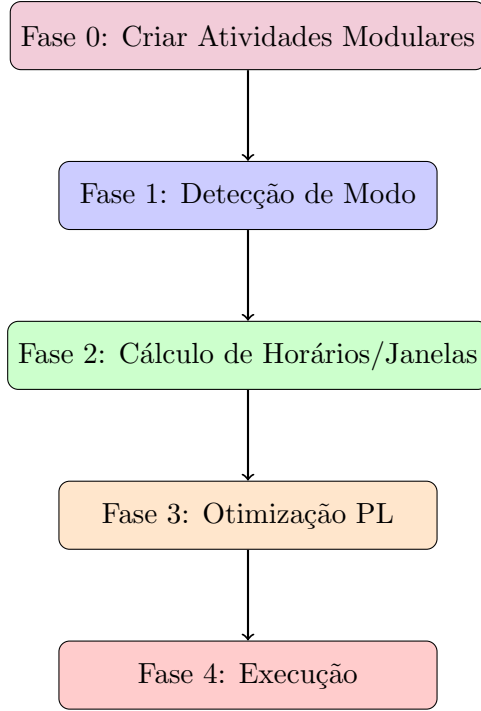
- Não otimiza ordem de execução
- Pode violar deadlines desnecessariamente
- Ignora conflitos de equipamentos até a execução
- Não maximiza utilização de recursos

## 3 Otimizador PL

O Otimizador PL é um sistema de otimização de produção que utiliza **Programação Linear Inteira Mista (MILP)** para determinar a ordem ótima de execução de pedidos.

### 3.1 Arquitetura do Otimizador PL

O sistema é composto por 5 fases principais:



### 3.2 Componentes do Otimizador PL

| Módulo                                              | Função                                |
|-----------------------------------------------------|---------------------------------------|
| <code>executor.py</code>                            | Orquestrador principal                |
| <code>detector_modos.py</code>                      | Detecta modo de otimização            |
| <code>calculador_horarios_deterministicos.py</code> | Cálculo backward scheduling           |
| <code>calculador_janelas.py</code>                  | Calcula janelas flexíveis             |
| <code>modelo_pl_ordenacao.py</code>                 | Modelo PL com OR-Tools CP-SAT         |
| <code>aplicador_ordenacao.py</code>                 | Executa pedidos (modo determinístico) |
| <code>aplicador_flexivel.py</code>                  | Executa pedidos (modo flexível)       |

Tabela 2: Módulos do Otimizador PL

### 3.3 Conceito de $\tau_{max}$ (tempo\_maximo\_de\_espera)

O parâmetro  $\tau_{max}$  define o gap máximo permitido entre atividades consecutivas:

- $\tau_{max} = 0$ : Atividades consecutivas sem gap (modo DETERMINÍSTICO)
- $\tau_{max} > 0$ : Gap de até  $\tau_{max}$  permitido entre atividades (modo FLEXÍVEL)

O modo é detectado automaticamente na Fase 1, após a criação das atividades modulares.

## 4 Formulação Matemática

### 4.1 Variáveis de Decisão

O modelo utiliza variáveis binárias para representar a seleção de pedidos e suas posições:

$$x_{p,j} \in \{0, 1\} \quad (1)$$

Onde:

- $p$  = índice do pedido ( $p \in P$ )
- $j$  = índice da janela temporal candidata ( $j \in J_p$ )
- $x_{p,j} = 1$  significa que o pedido  $p$  será executado na janela  $j$

Para o modelo de ordenação, também são utilizadas:

$$pos_i \in \{0, 1, \dots, n - 1\} \quad (2)$$

Onde  $pos_i$  representa a posição do pedido  $i$  na ordem de execução.

## 4.2 Função Objetivo

O objetivo é maximizar o número total de pedidos atendidos:

$$\max \sum_{p \in P} \sum_{j \in J_p} x_{p,j} \quad (3)$$

## 4.3 Restrições

### 4.3.1 Restrição 1: Unicidade por Pedido

Cada pedido utiliza no máximo uma janela temporal:

$$\sum_{j \in J_p} x_{p,j} \leq 1, \quad \forall p \in P \quad (4)$$

### 4.3.2 Restrição 2: Permutação Única (Ordenação)

Para o modelo de ordenação, todas as posições devem ser diferentes:

$$pos_i \neq pos_j, \quad \forall i \neq j \quad (5)$$

Esta restrição é implementada usando a constraint `AddAllDifferent` do OR-Tools.

### 4.3.3 Restrição 3: Tempo Máximo de Espera

Para atividades consecutivas:

$$0 \leq t_{início}^{(a+1)} - t_{fim}^{(a)} \leq \tau_{max} \quad (6)$$

Quando  $\tau_{max} = 0$ , isso garante atividades contíguas (modo DETERMINÍSTICO).

#### 4.3.4 Restrição 4: Conflitos Temporais (Equipamentos)

Se duas janelas se sobrepõem no tempo, não podem ser selecionadas simultaneamente:

$$x_{p_1, j_1} + x_{p_2, j_2} \leq 1 \quad (7)$$

Para todo par  $(p_1, j_1), (p_2, j_2)$  onde:

$$[t_1^{ini}, t_1^{fim}] \cap [t_2^{ini}, t_2^{fim}] \neq \emptyset \quad (8)$$

#### 4.3.5 Restrição 5: Deadline Rígido

Para pedidos com deadline obrigatório:

$$x_{p,j} = 0, \quad \forall j : |t_j^{fim} - d_p| > \epsilon \quad (9)$$

Onde  $d_p$  é o deadline do pedido  $p$  e  $\epsilon = 1$  minuto (tolerância).

## 5 Backward Scheduling

### 5.1 Modo Determinístico ( $\tau_{max} = 0$ )

Para o modo determinístico, o sistema utiliza **backward scheduling** para calcular horários fixos:

---

#### Algorithm 2 Backward Scheduling (Determinístico)

---

```

1:  $t_{atual} \leftarrow deadline$ 
2: for cada atividade  $a$  em ordem reversa do
3:    $t_{fim}^{(a)} \leftarrow t_{atual}$ 
4:    $t_{inicio}^{(a)} \leftarrow t_{fim}^{(a)} - duracao_a$ 
5:    $t_{atual} \leftarrow t_{inicio}^{(a)}$ 
6: end for

```

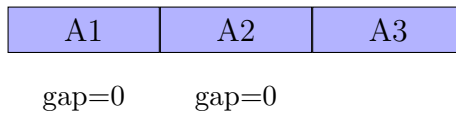
---

### 5.2 Modo Flexível ( $\tau_{max} > 0$ )

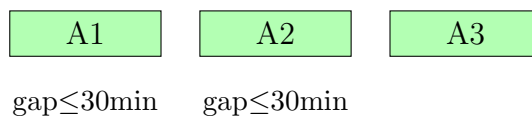
Para o modo flexível, o sistema calcula janelas temporais:

### 5.3 Representação Visual

Modo DETERMINÍSTICO ( $\tau_{max} = 0$ ):



Modo FLEXÍVEL ( $\tau_{max} = 30min$ ):





---

**Algorithm 3** Cálculo de Janelas Flexíveis

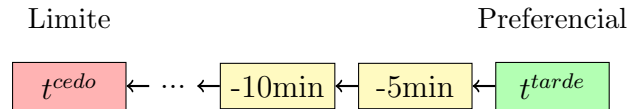
---

```
1:  $t_{fim}^{tarde} \leftarrow deadline$ 
2: for cada atividade  $a$  em ordem reversa do
3:   if  $a$  é última atividade then
4:      $t_{fim\ a}^{cedo} \leftarrow deadline$ 
5:      $t_{fim\ a}^{tarde} \leftarrow deadline$ 
6:   else
7:      $t_{fim\ a}^{tarde} \leftarrow t_{fim}^{tarde}$ 
8:      $t_{fim\ a}^{cedo} \leftarrow t_{fim}^{tarde} - \tau_{sucessora}^{max}$ 
9:   end if
10:   $t_{inicio\ a}^{tarde} \leftarrow t_{fim\ a}^{tarde} - duracao_a$ 
11:   $t_{inicio\ a}^{cedo} \leftarrow t_{fim\ a}^{cedo} - duracao_a$ 
12:   $t_{fim}^{cedo} \leftarrow t_{inicio\ a}^{cedo}$ 
13: end for
```

---

## 5.4 Estratégia de Retry (Modo Flexível)

Quando uma alocação falha no horário preferencial, o sistema tenta horários anteriores em steps de 5 minutos:



## 6 Complexidade Computacional

| Componente              | Complexidade                   |
|-------------------------|--------------------------------|
| Variáveis de decisão    | $O(n \cdot k)$                 |
| Restrições de unicidade | $O(n)$                         |
| Restrições de conflito  | $O(n^2 \cdot k^2)$ (pior caso) |

Tabela 3: Complexidade por componente ( $n$  = pedidos,  $k$  = janelas/pedido)

O solver OR-Tools CP-SAT utiliza técnicas avançadas de propagação de restrições e busca para encontrar soluções ótimas ou viáveis dentro do timeout configurado (padrão: 600 segundos).

## 7 Comparação: Sequencial vs Otimizado

## 8 Implementação

### 8.1 Exemplo de Uso - Sequencial

```
from services.gestores.producao import ExecutorPedidos
```

| Aspecto                                  | Sequencial | Otimizado PL        |
|------------------------------------------|------------|---------------------|
| Ordem de execução                        | Fixa       | Otimizada           |
| Conflitos de equipamentos                | Ignora     | Modela              |
| Modo DETERMINÍSTICO ( $\tau_{max} = 0$ ) | -          | Sim                 |
| Modo FLEXÍVEL ( $\tau_{max} > 0$ )       | -          | Sim                 |
| Deteção automática de modo               | Não        | Sim                 |
| Janelas flexíveis                        | Não        | Sim (modo FLEXÍVEL) |
| Retry na alocação                        | Não        | Sim (modo FLEXÍVEL) |
| Função objetivo                          | Nenhuma    | Max pedidos         |
| Método de solução                        | Greedy     | MILP                |
| Complexidade                             | $O(n)$     | $O(n^2 \cdot k^2)$  |

Tabela 4: Comparação entre métodos

```
# Criar executor
executor = ExecutorPedidos()

# Executar pedidos sequencialmente
sucesso = executor.executar_sequencial(pedidos_convertidos)

# Verificar estatísticas
stats = executor.obter_estatísticas()
print(f"Executados: {stats['pedidos_executados']}")
print(f"Com erro: {stats['pedidos_com_erro']}")
```

## 8.2 Exemplo de Uso - Otimizador PL

```
from otimizador import Executor

# Criar executor
executor = Executor()
executor.inicializar()

# Otimizar pedidos (detecta modo automaticamente)
solucao = executor.otimizar_pedidos(pedidos)

# Verificar resultados
print(f"Modo detectado: {solucao.modo_detectado}")
print(f"Pedidos atendidos: {solucao.pedidos_atendidos}")
print(f"Taxa de sucesso: {solucao.estatísticas['taxa_sucesso']}%")
```

## 8.3 Parâmetros de Configuração

- **timeout\_segundos:** Tempo máximo para resolução (padrão: 600s)
- **inicio\_jornada:** Horário de início da jornada de trabalho

## 9 Quando Usar Cada Algoritmo

### 9.1 Use o Algoritmo Sequencial quando:

- Os pedidos não possuem deadlines rígidos
- A ordem de chegada é importante para o cliente
- O número de pedidos é pequeno ( $i \leq 10$ )
- Simplicidade é mais importante que otimização
- Recursos computacionais são limitados

### 9.2 Use o Otimizador PL quando:

- Existem deadlines rígidos a serem respeitados
- Há conflitos potenciais de equipamentos
- É necessário maximizar o número de pedidos atendidos
- O número de pedidos é grande ( $i \geq 10$ )
- As atividades podem ter  $\tau_{max} > 0$  (gaps permitidos)
- É necessário flexibilidade na alocação temporal
- O sistema deve adaptar-se a conflitos de recursos

## 10 Conclusão

O sistema SIVIRA oferece dois algoritmos para alocação de pedidos:

1. O **Algoritmo Sequencial** é ideal para cenários simples, onde a ordem de chegada deve ser respeitada e não há restrições temporais complexas. Sua simplicidade garante baixo custo computacional e comportamento previsível.
2. O **Otimizador PL** oferece uma abordagem matematicamente fundamentada para o problema de scheduling de produção, suportando ambos os modos:
  - **DETERMINÍSTICO** ( $\tau_{max} = 0$ ): Atividades contíguas, sem gaps
  - **FLEXÍVEL** ( $\tau_{max} > 0$ ): Janelas temporais com retry automático

O modo é detectado automaticamente antes da execução, garantindo comportamento correto em ambos os casos.

A arquitetura modular permite fácil extensão para novos modos de operação e integração com o sistema de menu existente.